

Vertex Cover: Approximation Algorithm Design, Evaluation, and Visualization

Week 4 report for the greedy 2-approximation implemented in `PythonVisualizations/VertexCoverApprox.py`, its unit tests, and the event-driven Tk integration in `PythonVisualizations/Graph.py`.

Submission Metadata

- Course project deliverable: Week 4 — Approximation Algorithm Design and Visualization
- NP-complete problem: Minimum Vertex Cover
- Approximation algorithm: Greedy edge-selection 2-approximation
- Final submitted format: PDF
- Author: Anh Quoc Bui
- Date: 8/12/2026

1. Algorithm Design Summary

1.1 Problem Definition

Input: an undirected, simple graph $G = (V, E)$ (no self-loops, no weights).

Output: the smallest possible subset $C \subseteq V$ such that every edge in E has at least one endpoint in C . C is called a *vertex cover*.

The public snapshot type mirrors the one used for A* in Week 3:

python

```
@dataclass(frozen=True)
class GraphSnapshot:
    vertices: tuple[Node, ...]
    edges: tuple[tuple[Node, Node], ...] # each undirected edge once
```

`greedy_vertex_cover` validates that vertices are unique and hashable, that every edge references a known vertex, and that no edge is a self-loop (a self-loop cannot be "covered" by picking one endpoint in the usual sense, so it is rejected as invalid input rather than silently mishandled). A successful call returns a `VertexCoverResult` with the cover, the number of edges the graph started with, how many greedy iterations ran, and the complete immutable event trace.

1.2 Description of the NP-Complete Problem

Vertex Cover is one of Karp's original 21 NP-complete problems. Checking *whether a given set is a valid cover* is easy — walk every edge once and confirm one endpoint is marked, which is $O(V + E)$ — so the problem is in NP. Finding the *smallest* cover, however, has no known polynomial-time algorithm for general graphs, and Vertex Cover is NP-complete via a direct correspondence with **Independent Set** and

Clique: a set S is an independent set if and only if $V - S$ is a vertex cover, and it is a maximum independent set if and only if $V - S$ is a *minimum* vertex cover. Since Independent Set is NP-complete, so is Vertex Cover.

Why an exact optimum is hard: the only general-purpose exact method is to search the space of candidate subsets. A brute-force search tries every subset size from $\emptyset$ up to $|V|$ and checks feasibility, which costs $O(2^{|V|} \cdot |E|)$ in the worst case — completely impractical once $|V|$ passes about 25–30 vertices. Exact branch-and-bound solvers exist and do much better in practice, but they remain exponential in the worst case.

Trade-off between optimality and efficiency: the approximation algorithm implemented here trades a *provable* factor-of-2 optimality gap for a guaranteed $O(V + E)$ running time — a difference between "always fast" and "sometimes intractable."

1.3 The Approximation Algorithm and How It Works

The implementation uses the standard **greedy edge-selection (maximal matching) 2-approximation**:

1. Put every edge in a queue, in a fixed deterministic order.
2. Repeatedly take the next edge (u, v) .
 - If both endpoints are already covered (the edge was already satisfied by an earlier pick), skip it.
 - Otherwise add **both** u and v to the cover, and discard every other edge touching u or v — they are now automatically covered.
3. Stop when no edges remain.

Why this is a 2-approximation. Every edge the algorithm *selects* is, by construction, disjoint from every previously selected edge (its endpoints could not already be in the cover, or it would have been discarded in step 2). So the selected edges form a **matching** M . Any valid vertex cover — including the optimal one, OPT — must contain at least one endpoint from each edge in M , and those endpoints must all be distinct because the edges in M share no vertices. Therefore $|OPT| \geq |M|$. The greedy algorithm's cover contains exactly two vertices per matched edge, so $|cover| = 2|M| \leq 2|OPT|$. This is a short, complete proof, not an empirical claim, and it holds for every input graph.

1.4 Key Design Decisions

- **Immutable snapshot + event trace**, matching the architecture already proven for A* search (AStarSearch.py): `greedy_vertex_cover` never imports Tkinter and never mutates its input; a visualization consumes a synchronous stream of frozen `VertexCoverEvent` records instead of re-deriving decisions from scratch.
- **Deque with lazy skip** instead of repeatedly scanning the remaining edge set for the "next" edge. This is the same idiom `a_star_search` uses to skip stale heap entries, and it keeps the algorithm $O(V + E)$ rather than $O(V \cdot E)$.
- **Canonicalizing bidirectional edges.** The Tk Graph widget stores both (u, v) and (v, u) when "bidirectional" is enabled; the adapter (`snapshot_graph`) collapses that pair into one undirected edge before the algorithm ever sees it, so the cover computation only ever reasons about undirected edges.

- **Rejecting self-loops explicitly** with a clear `ValueError` rather than either crashing obscurely or silently covering them, keeping correctness guarantees intact for the graphs the algorithm claims to support.

1.5 Evidence of Using the Guiding Questions

Understanding the problem — see §1.1–1.2. A hand-solvable example: the path graph A–B–C–D has three edges. Manually, {B, C} covers all three edges (A–B via B, B–C via either, C–D via C), and no single vertex can, so the true optimum is 2.

Exploring simple approaches — brute force costs $O(2^{|V|} \cdot |E|)$ (§1.2). A cheaper heuristic is exactly the greedy strategy in §1.3; another common heuristic (not implemented here) is an LP relaxation of the integer program followed by rounding, which also yields a factor-2 bound but needs a general LP solver rather than a simple queue.

Relating to known problems — Vertex Cover is dual to Independent Set and Clique (§1.2), and is the simplest special case of the more general **Set Cover** problem (each edge is a 2-element "set" that must be hit). Set Cover's greedy approximation (repeatedly pick the set covering the most uncovered elements) gives an $O(\log n)$ factor, which is *worse* than the 2-approximation available here specifically because Vertex Cover's sets always have size 2 — a structural special case worth calling out explicitly, and the reason Vertex Cover has its own, tighter approximation algorithm instead of just using the general Set Cover one.

Special cases and simplifications — on **bipartite graphs**, König's theorem says the minimum vertex cover size exactly equals the maximum matching size, so an *exact* polynomial algorithm exists there (via maximum bipartite matching) instead of only an approximation. On **trees**, a greedy leaf-driven algorithm also finds the exact optimum in linear time. The general-graph greedy 2-approximation implemented here does not special-case either shape; on a triangle (§2.3) it happens to still find the true optimum, but on the path example above it does not (§2.3).

Algorithm design paradigms — this is a **greedy** algorithm: each step makes the locally cheapest possible commitment (cover one uncovered edge at minimum vertex cost, 2 vertices) without reconsidering earlier choices. It relies on a simple adjacency-set data structure (§1.4) rather than local search, iterative improvement, or randomization.

Evaluation considerations — see §2.

2. Implementation and Evaluation

2.1 Architecture

text

```
Graph instance (Tk, unweighted, undirected)
-> snapshot_graph
-> immutable GraphSnapshot
-> greedy_vertex_cover
-> synchronous immutable VertexCoverEvent stream
-> immutable VertexCoverResult
-> Tk event renderer and final result overlay in Graph.py
```

VertexCoverApprox.py imports no Tk modules. The pure algorithm receives no widget, canvas item ID, animation state, or framework call environment — it is fully testable by import alone.

2.2 Event Contract

python

```
class VertexCoverEventKind(str, Enum):
    SEARCH_STARTED = "search_started"
    EDGE_ALREADY_COVERED = "edge_already_covered"
    EDGE_SELECTED = "edge_selected"
    VERTEX_ADDED = "vertex_added"
    EDGES_ELIMINATED = "edges_eliminated"
    SEARCH_COMPLETE = "search_complete"
```

Each event carries a fully denormalized snapshot (cover, remaining_edges, total_edges) so a renderer never has to re-derive algorithm state — the same design choice A* search made for its frontier and closed-set snapshots.

2.3 Correctness Evidence

tests/test_vertex_cover.py (16 tests, all passing) verifies:

- the adapter correctly collapses a bidirectional (u, v)/(v, u) pair into one undirected edge, and rejects unknown-vertex or self-loop edges;
- an empty graph produces an empty, valid cover;
- **the path graph A–B–C–D manual example:** the algorithm greedily picks (A, B) first, which is disjoint from (C, D), so it ends up with the full cover {A, B, C, D} (size 4) against a brute-force-proven optimum of 2 — landing *exactly* on the theoretical 2× worst case and demonstrating an EDGE_ALREADY_COVERED skip event ((B, C) is already satisfied once B is in the cover);
- **the triangle example** A–B, B–C, A–C: greedy picks {A, B}, which happens to already cover all three edges, matching the brute-force optimum of 2 exactly (ratio 1.0);
- **feasibility on 40 deterministic seeded random graphs** (up to 7 vertices, densities varied): every edge has at least one endpoint in the returned cover, and no vertex is ever added to the cover twice;
- **the 2-approximation bound against brute force:** for 25 additional seeded random graphs (≤ 7 vertices, so exhaustive search is feasible), $\text{len}(\text{greedy cover}) \leq 2 \times (\text{brute-force optimum})$ holds in every case;
- the selected edges always form a matching (pairwise disjoint), and the cover size is always exactly twice the number of selected edges;
- event sequence numbers are contiguous and events fire in the documented order; running the same snapshot twice produces byte-identical results.

verify_vertex_cover_visualization.py additionally exercises the live Tk integration: button enable/disable rules, the guard messages for "no edges" and "directed graph," the rendered status text, cleanup of transient renderer items, persistence and replacement of the final highlighted-cover overlay, and propagation of a simulated user stop and a simulated renderer failure without leaking canvas items or leaving the operation mutex locked.

Verified commands (all pass with OK):

powershell

```
python -m py_compile PythonVisualizations/VertexCoverApprox.py
PythonVisualizations/Graph.py verify_vertex_cover_visualization.py
python -m unittest tests.test_vertex_cover
python verify_vertex_cover_visualization.py
```

2.4 Efficiency Evaluation

Every edge is enqueued once and dequeued once; each dequeue does an $O(1)$ membership check against the remaining edge set (a dict keyed by `frozenset({u, v})`), and each vertex added to the cover triggers removal of its incident edges from that same structure, work that is charged once per edge total (not once per vertex). This gives **$O(V + E)$ time and $O(V + E)$ space** — asymptotically identical to the cost of just reading the graph once, and dramatically cheaper than the $O(2^V \cdot E)$ brute-force alternative from §1.2.

For context, this simple combinatorial bound is close to the best known for a problem this general: it is known that no polynomial algorithm can guarantee better than roughly a $1.36\times$ approximation unless $P = NP$ (Dinur–Safra), and even the more powerful LP-relaxation-and-rounding technique only reaches the same factor of 2 that this greedy method achieves with far less machinery.

2.5 Ease-of-Use Evaluation

The public surface is two functions (`snapshot_graph`, `greedy_vertex_cover`) and four small dataclasses/enums — no configuration beyond the graph itself. Inside the Tk app, the feature is a single "Vertex Cover (Approx)" button on the existing unweighted Graph visualization, enabled only when the graph has edges and is in bidirectional (undirected) mode, following the exact enable/disable idiom already used by Depth-first Traverse, Minimum Spanning Tree, and Topological Sort. It participates in the same animation-speed, pause/resume, single-step, and stop controls as every other operation with no bespoke UI code required.

3. Visualization Description

3.1 Visualization Goal

The Tk layer consumes the immutable event stream to show, step by step, which edge the greedy algorithm is currently considering, which vertices get added to the cover and when, which edges become satisfied ("eliminated") as a side effect, and which edges are skipped because they were already satisfied by an earlier pick. It does not re-decide anything — every highlight is a direct rendering of one `VertexCoverEvent`.

3.2 Framework Integration Points

Integration lives entirely in `PythonVisualizations/Graph.py` (the unweighted-graph visualization), mirroring how A* was added to `WeightedGraph.py`:

- `GraphVertexCoverAdapter` is the typed boundary between the live Tk graph and `snapshot_graph`;

- `VertexCoverEventRenderer` draws non-destructive overlay items (an oval per cover vertex, a reusable line for the edge currently under consideration, and lines for edges the cover has already satisfied) instead of mutating the base vertex/edge canvas items, so no color needs to be remembered and restored;
- a small status-text overlay reports `cover size` and `edges remaining: n/total` after every event;
- "`Vertex Cover (Approx)`" is a new button beside the other graph-wide operations, enabled only when the graph has at least one edge and bidirectional edges are enabled;
- `createCallEnvironment`, `highlightCode`, `wait`, and normal `cleanUp` preserve every existing framework control (speed, pause, step, stop);
- the final cover is redrawn as a persistent tagged overlay (`VERTEX_COVER_RESULT_TAG`) after the transient event items are cleaned up, exactly as `AStarEventRenderer`'s result path is; `Graph.cleanUp` is overridden to delete that tag first, so the next operation (or the next cover computation) always starts from a clean canvas.

No changes to `GraphBase.py` or `VisualizationApp.py` were required.

3.3 Visual Encoding

State	Implemented representation
Edge under consideration	Thick blue overlay line
Edge already covered (skipped)	Thick gray overlay line, briefly shown
Vertex added to cover	Orange (<code>chocolate2</code>) ring overlay around the vertex, kept for the run
Edge eliminated by a pick	Persistent gray overlay line for the rest of the run
Status readout	Text showing <code>cover size: n</code> and <code>edges remaining: r/total</code>
Final cover	Orange ring overlays tagged <code>VertexCoverResult</code> , kept until next run
Guard rails	Status-bar message when there are no edges or the graph is directed

Transient items use the `VertexCover` canvas tag and the operation's call environment, so they are removed automatically by `cleanUp`. The persistent final-result overlay uses its own tag and survives until the next operation clears it.

3.4 Planned User Workflow

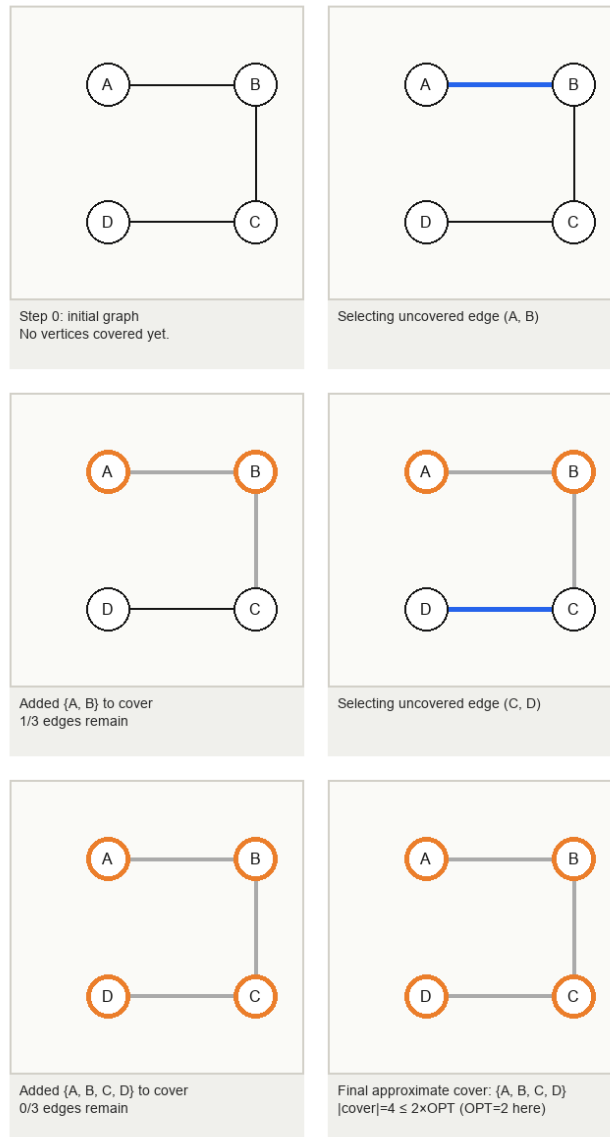
4. Open `Graph` (unweighted) directly, or through `python DatastructureVisualizations.py`.

5. Create or randomly fill an undirected graph (bidirectional edges enabled, which is the graph's default).
6. Click `Vertex Cover (Approx)`.
7. Observe each edge being selected, its endpoints joining the cover, and the edges that selection satisfies being grayed out, until no edges remain.
8. Use the speed, pause/resume, step, and stop controls exactly as with any other operation.
9. Read the final message, which reports the cover size and its members, and see the cover highlighted on the graph.

3.5 Example Run

The figure below was generated directly from the real algorithm trace (not hand-drawn) on the path graph A–B–C–D, the same fixture used by `verify_vertex_cover_visualization.py`. It shows the initial graph, the edge-selection and elimination steps, and the final cover landing exactly on the $2\times$ optimal worst case described in §2.3.

Greedy 2-Approximation Vertex Cover — Path Graph A-B-C-D



Greedy vertex cover step-by-step trace on a path graph

3.6 Launcher Integration

`Graph` remains the only visualization class discovered by the launcher. `VertexCoverApprox` contains no `VisualizationApp` subclass and does not appear as a separate launcher entry — canonical discovery verifies exactly one class identified as `PythonVisualizations.Graph.Graph`, the same pattern already established for `ASearch/WeightedGraph`.

4. Verification

Verification commands, run from the repository root:

powershell


```
python -m py_compile PythonVisualizations/VertexCoverApprox.py
PythonVisualizations/Graph.py verify_vertex_cover_visualization.py
python -m unittest tests.test_vertex_cover
python verify_vertex_cover_visualization.py
python PythonVisualizations/Graph.py
python DatastructureVisualizations.py
```

Observed result: all tests/test_vertex_cover.py cases pass (16 passed under pytest -q), and verify_vertex_cover_visualization.py reports OK for the live Tk integration, alongside the unchanged, still-passing Week 2 and Week 3 verification commands (verify_priority_queue.py, verify_astar_visualization.py, tests/test_astar_search.py).